EarlyCare AI

Source Code Documentation

Includes dependency installation, model training for four dataset types, a reusable prediction function, and a FastAPI endpoint for diabetes screening.

Important: This is an experimental educational prototype. Its output is not a medical diagnosis and must not be used for clinical decisions.

1. Install Dependencies

```
pip install pandas numpy scikit-learn xgboost joblib shap fastapi uvicorn
```

2. Train Models — train_models.py

```
import os
import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# Create a folder to save trained models
os.makedirs("models", exist_ok=True)

# Dataset configuration
DATASETS = {
    "diabetes": {
        "file": "data/diabetes.csv",
        "target": "Outcome"
    },
    "heart": {
        "file": "data/heart.csv",
        "target": "target"
    },
    "hypertension": {
        "file": "data/hypertension.csv",
        "target": "target"
    },
    "kidney": {
        "file": "data/kidney.csv",
        "target": "classification"
    }
}


def train_model(name, file_path, target_column):
    print(f"\\nTraining {name} model...")

    # Load dataset
    df = pd.read_csv(file_path)
    df = df.dropna(subset=[target_column])

    X = df.drop(columns=[target_column])

    y = df[target_column]

    # Convert categorical columns into numeric columns
    X = pd.get_dummies(X, dummy_na=True)

    # Convert target labels to binary values where necessary.
    # Check the mapping against your dataset's documentation.
    if name == "kidney":
        y = (
            y.astype(str)
             .str.strip()
             .str.lower()
             .map({
                 "ckd": 1,
                 "notckd": 0,
                 "not ckd": 0
             })
        )
        valid = y.notna()
        X = X.loc[valid]
        y = y.loc[valid].astype(int)
    else:
        y = pd.to_numeric(y, errors="coerce")
        valid = y.notna()
        X = X.loc[valid]
        y = y.loc[valid].astype(int)

    if set(y.unique()) != {0, 1}:
        raise ValueError(
            f"{name}: target must contain both 0 and 1. "
            f"Found {sorted(y.unique())}"
        )
```

```
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    model = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("classifier", RandomForestClassifier(
            n_estimators=200,
            random_state=42,
            class_weight="balanced"
        ))
    ])

    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print(f"\\n{name.upper()} RESULTS")
    print("Accuracy:", accuracy_score(y_test, predictions))
    print("Precision:", precision_score(y_test, predictions, zero_division=0))
    print("Recall:", recall_score(y_test, predictions, zero_division=0))
    print("F1 Score:", f1_score(y_test, predictions, zero_division=0))
    print("\\nClassification Report:")
    print(classification_report(y_test, predictions, zero_division=0))

    artifact = {
        "model": model,
        "features": X.columns.tolist()
    }
    joblib.dump(artifact, f"models/{name}_model.joblib")
    print(f"Saved models/{name}_model.joblib")


if __name__ == "__main__":
    for name, config in DATASETS.items():
        if not os.path.exists(config["file"]):
            print(f"Skipping {name}: dataset not found at {config['file']}")
            continue
        try:
            train_model(name, config["file"], config["target"])
        except Exception as error:
            print(f"Could not train {name}: {error}")
```

3. Generic Prediction Function — predict_disease.py

```
import joblib
import pandas as pd


def predict_disease(disease, patient_data):
    model_path = f"models/{disease}_model.joblib"
    artifact = joblib.load(model_path)

    model = artifact["model"]
    features = artifact["features"]

    input_df = pd.DataFrame([patient_data])
    input_df = pd.get_dummies(input_df, dummy_na=True)
    input_df = input_df.reindex(columns=features, fill_value=0)

    probabilities = model.predict_proba(input_df)[0]
    classes = model.named_steps["classifier"].classes_
    positive_index = list(classes).index(1)
    risk_score = probabilities[positive_index] * 100

    return {
        "disease": disease,
        "estimated_model_score_percent": round(float(risk_score), 2),
        "message": (
            "Experimental model score only. "
            "Not a diagnosis or a validated clinical risk estimate."
        )
    }


if __name__ == "__main__":
    sample_patient = {
        "Pregnancies": 1,
        "Glucose": 120,
        "BloodPressure": 70,
        "SkinThickness": 20,
        "Insulin": 79,
        "BMI": 25.0,
        "DiabetesPedigreeFunction": 0.3,
        "Age": 30
    }

    result = predict_disease("diabetes", sample_patient)
    print(result)
```

4. FastAPI Service — app.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import joblib
import pandas as pd
import os

app = FastAPI(
    title="EarlyCare AI",
    description="Experimental chronic disease risk screening prototype",
    version="1.0"
)


class PatientInput(BaseModel):
    age: int = Field(ge=18, le=120)
    bmi: float = Field(gt=0, le=100)
    glucose: float = Field(gt=0, le=2000)
    blood_pressure: float = Field(gt=0, le=500)


@app.get("/")
def home():
    return {
        "project": "EarlyCare AI",
        "status": "running",
        "purpose": "Educational screening prototype"
    }


@app.post("/predict/diabetes")
def predict_diabetes(patient: PatientInput):
    model_path = "models/diabetes_model.joblib"

    if not os.path.exists(model_path):
        raise HTTPException(
            status_code=503,
            detail="Diabetes model has not been trained."
        )

    artifact = joblib.load(model_path)
    model = artifact["model"]
    features = artifact["features"]

    # These names must match the training dataset.
    input_data = {
        "Pregnancies": 0,
        "Glucose": patient.glucose,
        "BloodPressure": patient.blood_pressure,

        "SkinThickness": 0,
        "Insulin": 0,
        "BMI": patient.bmi,
        "DiabetesPedigreeFunction": 0,
        "Age": patient.age
    }

    X = pd.DataFrame([input_data])
    X = X.reindex(columns=features, fill_value=0)

    probabilities = model.predict_proba(X)[0]
    classes = model.named_steps["classifier"].classes_
    positive_index = list(classes).index(1)
    score = float(probabilities[positive_index])

    return {
        "model": "Random Forest",
        "disease": "diabetes",
        "experimental_score": round(score, 4),
        "disclaimer": (
            "Not a diagnosis. This experimental score is "
            "not validated for clinical decision-making."
        )
    }
```